Open in app ↗

Medium Search Play Games Write 3

Welcome Offer Access to everything. Now 30% off. Upgrade now

Data Engineering + Data Engineer + Databricks + Big Data + Apache Spark +

Implementing an Automated Undo Data Integration Job: A Step-by-Step Guide



Anurag Sharma 14 min read · Sep 7, 2026

2

How we built a production-grade recovery system that turns 3-day firefighting incidents into automated 5-minute recoveries



> ✅ *If you work with upstream systems and have a possibility of data corruption?*
>
> ✅ *If you manage thousands of tables across raw, processed, and curated layers of delta tables and dread the day corruption strikes again?*
>
> ✅ *If your on-call engineers wake up in cold sweats thinking about the next day incident response cycle?*
>
> ✅ *If you use Unity Catalog and want to leverage metadata for intelligent, automated recovery?*
>
> ✅ *If you believe your data platform should be resilient, self-healing, and require zero manual intervention during recovery?*
>
> *If ANY of the above questions is YES ✅ for you, then this blog is for you.*

Table of Contents

1. The Crisis That Started It All
2. Why Manual Fixes Don’t Scale
3. Architecture Design: Building the Recovery Engine
4. Step 1: Setting Up the Metadata Foundation
5. Step 2: Building the Corruption Detection Layer
6. Step 3: Implementing the Rollback Engine
7. Step 4: Adding Integrity Validation
8. Step 5: Creating the Orchestration & Triggering Layer
9. Step 6: Deployment & Monitoring
10. Lessons Learned & Key Takeaways

The Crisis That Started It All

Picture this: Tuesday morning, 9 AM. Teams explodes.

An upstream system deployed a minor version change. Routine stuff. It was supposed to be backward-compatible. But it wasn’t.

Our raw data layer — the foundation of everything downstream — was corrupted. Thousands of tables. Unusable data. Broken pipelines everywhere.

By Wednesday afternoon, our team had spent 36+ hours:

- Creating Excel spreadsheets to track 2,847 affected tables

- Manually identifying the last “good” version for each table
- Writing individual rollback queries using Delta Lake’s `-1` version operation
- Running queries one by one, then manually validating results
- Tracking progress in a shared Google Doc (not ideal for an engineering team)

By Thursday evening, we’d finally recovered. Exhausted. Burned out.

And the worst part? We all knew this could happen again. Because there was nothing *automated* preventing it.

That weekend, we made a decision: never again.

This blog documents exactly how we built an automated system to prevent those three-day nightmares from ever happening again.

Why Manual Fixes Don’t Scale

Before diving into the technical implementation, let’s be crystal clear about why we couldn’t just repeat this process next time:

The Math Doesn’t Work

- 3 days × 5 engineers = 120 hours of lost productivity
- For a company with a fully-loaded engineer cost of ~$50/hour, that’s $6,000 per incident
- We had two incidents in one month — that’s $12,000 just in labor costs

Risk Multiplies With Scale

- As table count grows, manual error rates increase
- Each custom query written is a chance to introduce bugs
- Manual verification is cognitively expensive — fatigue leads to missed edge cases

It's Not Reliable

- Different engineers make different decisions
- No audit trail of what was rolled back and when
- Impossible to debug if something goes wrong during recovery

It Breaks Team Morale

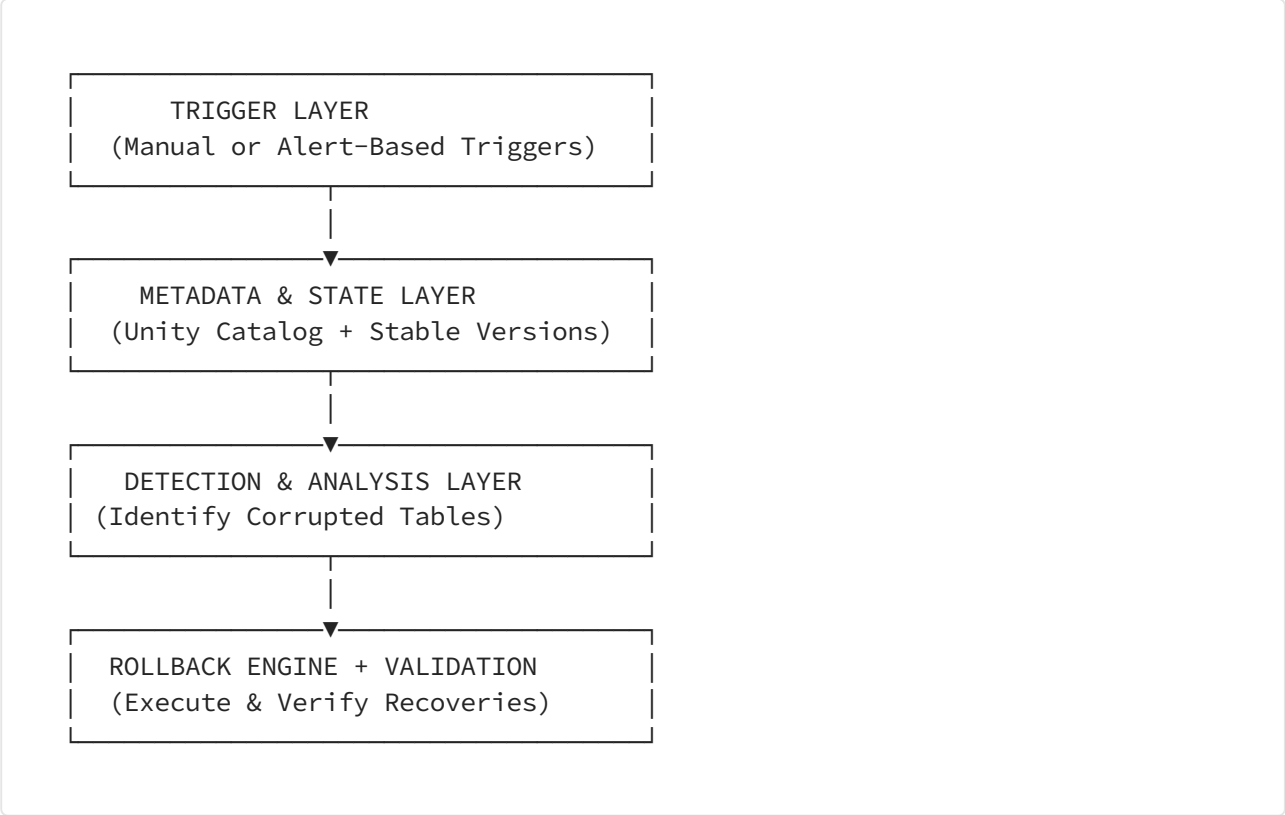
- Nobody wants to spend three days in a spreadsheet incident response
- On-call rotation becomes terrifying
- Engineers start looking for jobs at companies with better infrastructure

The decision to automate wasn't optional. It was essential.

Architecture Design: Building the Recovery Engine

Before we wrote a single line of code, we sketched out the system architecture.

We knew we needed four core components:

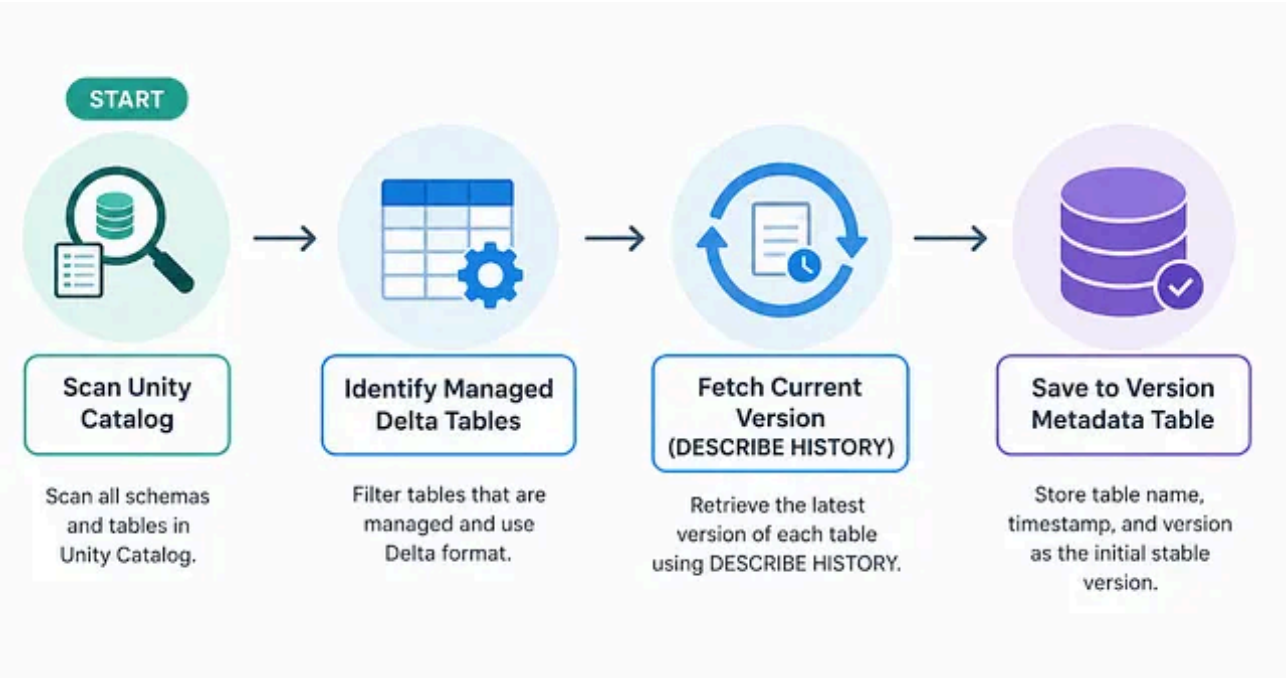


Each layer has a single responsibility, making the system testable and maintainable.

Step 1: Setting Up the Metadata Foundation

The Challenge: How do we know which version of each table is “good”? We can’t rely on manual spreadsheets or tribal knowledge. We needed a system of record.

The Solution: We created a metadata tracking table in Unity Catalog that records the stable version of every critical Delta table.



1.1: Create the Metadata Tracking Table

```
-- Create a metadata tracking table in Unity Catalog
CREATE TABLE IF NOT EXISTS system.data_quality.table_version_metadata (
    catalog_name STRING,
    schema_name STRING,
    table_name STRING,
    full_table_name STRING,
    current_version LONG,
    last_stable_version LONG,
    is_corrupted BOOLEAN,
    corruption_detected_at TIMESTAMP,
    last_recovery_at TIMESTAMP,
    recovery_status STRING,
    created_at TIMESTAMP,
    updated_at TIMESTAMP,
    created_by STRING
);

-- Create a unique index on full_table_name
CREATE UNIQUE INDEX idx_full_table_name
ON system.data_quality.table_version_metadata(full_table_name);
```

Why this approach:

- Centralized record of truth for all table versions
- Audit trail of corruption and recovery events
- Queryable by any downstream process

1.2: Populate Initial Stable Versions

When the system first starts, we need to bootstrap the metadata with current versions:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import current_timestamp, lit
import datetime

spark = SparkSession.builder.appName("MetadataBootstrap").getOrCreate()

# Get all Delta tables managed by Unity Catalog
tables_query = """
SELECT
 catalog_name,
 schema_name,
 table_name,
 CONCAT(catalog_name, '.', schema_name, '.', table_name) as full_table_name
FROM system.information_schema.tables
WHERE table_catalog = 'main'
  AND schema_name IN ('raw', 'processed', 'curated')
  AND table_type = 'EXTERNAL'
"""

tables_df = spark.sql(tables_query)

# For each table, get its current version
def get_current_version(full_table_name):
    try:
        version_result =
spark.sql(f"DESCRIBE HISTORY {full_table_name} LIMIT 1").collect()
        if version_result:
            return version_result[0].version
    except Exception as e:
        print(f"Error getting version for {full_table_name}: {str(e)}")
        return None

# Build metadata dataframe
metadata_rows = []
for row in tables_df.collect():
```

```
        full_table_name = row.full_table_name
        current_version = get_current_version(full_table_name)

        if current_version is not None:
            metadata_rows.append({
                "catalog_name": row.catalog_name,
                "schema_name": row.schema_name,
                "table_name": row.table_name,
                "full_table_name": full_table_name,
                "current_version": current_version,
                "last_stable_version": current_version,
                "is_corrupted": False,
                "corruption_detected_at": None,
                "last_recovery_at": None,
                "recovery_status": "STABLE",
                "created_at": datetime.datetime.now(),
                "updated_at": datetime.datetime.now(),
                "created_by": "bootstrap_job"
            })

    # Write to metadata table
    metadata_df = spark.createDataFrame(metadata_rows)
    metadata_df.write.mode("overwrite").saveAsTable(
        "system.data_quality.table_version_metadata"
    )

    print(f"Bootstrap complete: {len(metadata_rows)} tables cataloged")
```

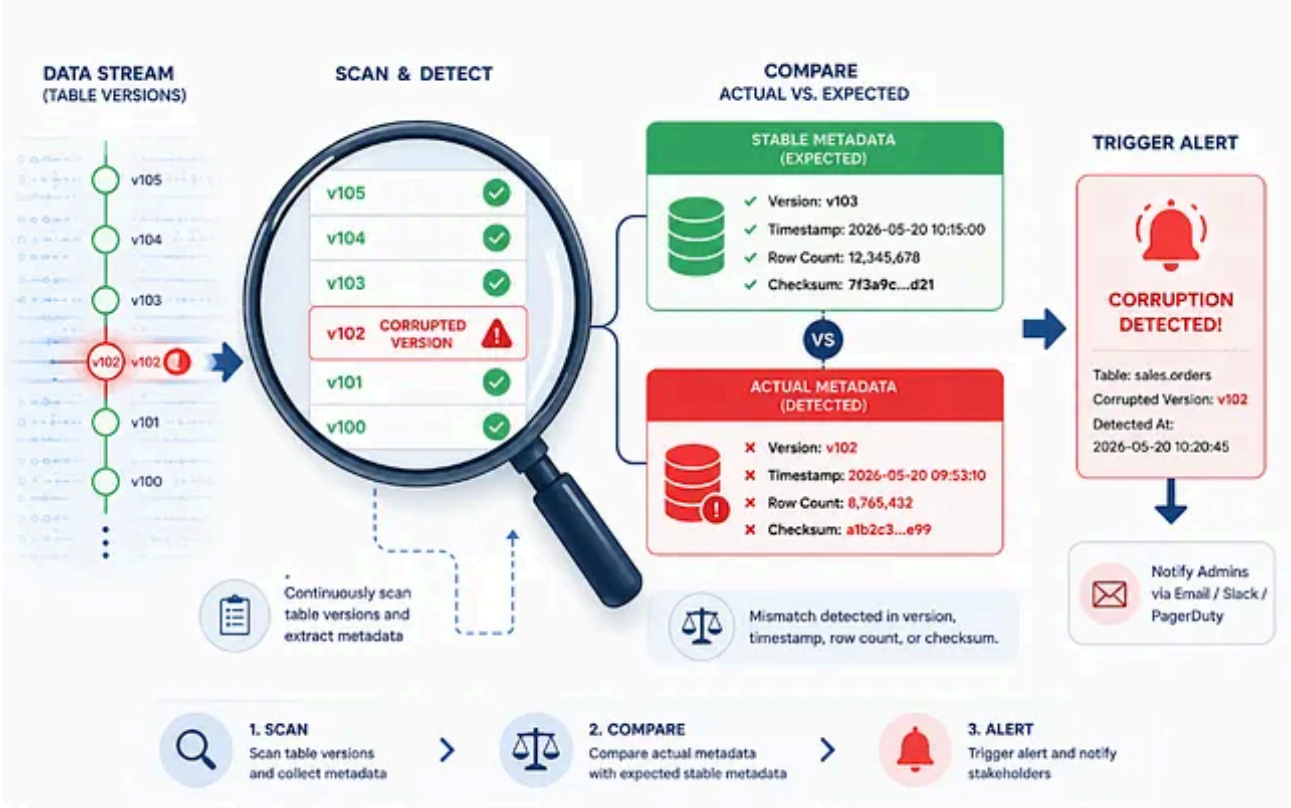
Key Point: This initial bootstrap runs once. After that, the metadata table is updated only when corruption is detected or recovery completes.

Step 2: Building the Corruption Detection Layer

The Challenge: How do we automatically detect when a table has been corrupted? We can't check every table manually. We need an intelligent detection system.



The Solution: We built a detection job that compares expected metadata state with actual Delta table state.



2.1: Create the Detection Job

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, current_timestamp
import json
from datetime import datetime

spark = SparkSession.builder.appName("CorruptionDetection").getOrCreate()

class CorruptionDetector:
```

```
    def __init__(self, spark):
        self.spark = spark
        self.metadata_table = "system.data_quality.table_version_metadata"
        self.detection_log_table = "system.data_quality.corruption_detection_log

    def detect_corruptions(self):
        """
        Compare metadata expectations with actual Delta table state.
        Flag tables as corrupted if versions diverge unexpectedly.
        """

        # Get all tracked tables from metadata
        metadata_df = spark.sql(f"SELECT * FROM {self.metadata_table}")

        corrupted_tables = []

        for row in metadata_df.collect():
            full_table_name = row.full_table_name
            expected_version = row.last_stable_version

            try:
                # Get actual current version
                actual_version = self._get_table_version(full_table_name)

                if actual_version is None:
                    continue

                # Check if version jumped unexpectedly
                # (more than 10 versions in < 1 hour suggests corruption)
                version_jump = actual_version - expected_version

                if version_jump > 10:
                    print(f"CORRUPTION DETECTED: {full_table_name}")
                    print(f"Expected version: {expected_version}, Actual: {actua

                    corrupted_tables.append({
                        "full_table_name": full_table_name,
                        "expected_version": expected_version,
                        "actual_version": actual_version,
                        "version_jump": version_jump,
                        "detection_time": datetime.now(),
                        "status": "PENDING_RECOVERY"
                    })

            except Exception as e:
                print(f"Error detecting corruption for {full_table_name}: {str(e
                continue

        # Log detection events
        if corrupted_tables:
            detection_df = spark.createDataFrame(corrupted_tables)
            detection_df.write.mode("append").saveAsTable(self.detection_log_tab
```

```
            # Update metadata to mark tables as corrupted
            for table_info in corrupted_tables:
                self._mark_table_corrupted(
                    table_info["full_table_name"],
                    table_info["expected_version"]
                )

        return corrupted_tables

    def _get_table_version(self, full_table_name):
        """Get current version of a Delta table"""
        try:
            history = spark.sql(f"DESCRIBE HISTORY {full_table_name} LIMIT 1").c
            if history:
                return history[0].version
        except Exception as e:
            print(f"Error: {str(e)}")
        return None

    def _mark_table_corrupted(self, full_table_name, last_stable_version):
        """Update metadata to mark table as corrupted"""
        update_query = f"""
        UPDATE {self.metadata_table}
        SET
            is_corrupted = true,
            corruption_detected_at = current_timestamp(),
            recovery_status = 'PENDING_RECOVERY',
            updated_at = current_timestamp()
        WHERE full_table_name = '{full_table_name}'
        """
        spark.sql(update_query)

# Run detection
detector = CorruptionDetector(spark)
corrupted = detector.detect_corruptions()

print(f"\n Detection complete: Found {len(corrupted)} corrupted tables")
```

Key Features:

- Compares expected vs. actual versions
- Detects anomalous version jumps (heuristic: >10 versions in short time)
- Logs all detection events for audit trail

- Marks corrupted tables in metadata

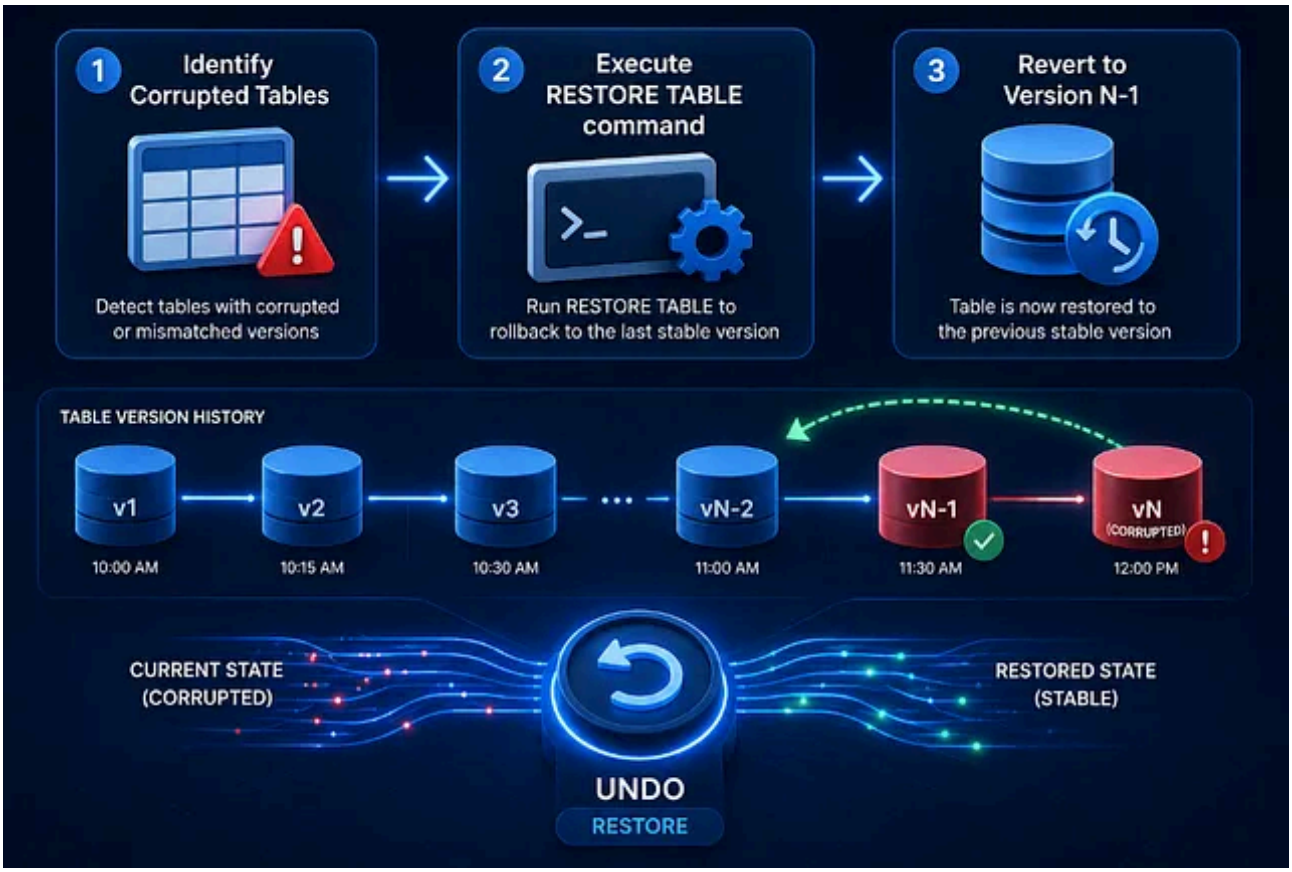
Why This Works: Delta Lake's version history is immutable. If a table went from version 100 to version 150 in 5 minutes due to a bad upstream change, that's not normal. We flag it.

Step 3: Implementing the Rollback Engine

The Challenge: Once we have identified corrupted tables, how do we safely roll them back? We need to:

- Revert each table to its last known-stable version
- Handle failures gracefully
- Track what we have done

The Solution: A rollback engine that uses Delta Lake's `RESTORE` command intelligently.



3.1: Build the Rollback Engine

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, current_timestamp
import logging
from datetime import datetime

spark = SparkSession.builder.appName("RollbackEngine").getOrCreate()

# Set up logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("RollbackEngine")

class RollbackEngine:
    def __init__(self, spark):
        self.spark = spark
        self.metadata_table = "system.data_quality.table_version_metadata"
        self.recovery_log_table = "system.data_quality.recovery_log"

    def rollback_corrupted_tables(self):
        """
        Find all corrupted tables and roll them back to last stable version.
        """
```

```
            # Get all corrupted tables
            corrupted_df = spark.sql(f"""
            SELECT
                full_table_name,
                current_version,
                last_stable_version
            FROM {self.metadata_table}
            WHERE is_corrupted = true AND recovery_status = 'PENDING_RECOVERY'
            """)

            corrupted_tables = [row for row in corrupted_df.collect()]

            logger.info(f" Starting rollback for {len(corrupted_tables)} tables")

            recovery_results = []

            for table in corrupted_tables:
                full_table_name = table.full_table_name
                target_version = table.last_stable_version

                logger.info(f"Rolling back {full_table_name} to version {target_vers

                try:
                    # Use Delta Lake's RESTORE command to revert to stable version
                    restore_query = f"""
                    RESTORE TABLE {full_table_name}
                    TO VERSION AS OF {target_version}
                    """

                    spark.sql(restore_query)

                    logger.info(f" Successfully rolled back {full_table_name}")

                    recovery_results.append({
                        "full_table_name": full_table_name,
                        "rollback_target_version": target_version,
                        "rollback_status": "SUCCESS",
                        "recovery_timestamp": datetime.now(),
                        "error_message": None
                    })

                except Exception as e:
                    logger.error(f" Rollback failed for {full_table_name}: {str(e)}"

                    recovery_results.append({
                        "full_table_name": full_table_name,
                        "rollback_target_version": target_version,
                        "rollback_status": "FAILED",
                        "recovery_timestamp": datetime.now(),
                        "error_message": str(e)
                    })
```

```
            # Log all recovery attempts
            if recovery_results:
                recovery_df = spark.createDataFrame(recovery_results)
                recovery_df.write.mode("append").saveAsTable(self.recovery_log_table

            # Summarize results
            successful = sum(1 for r in recovery_results if r["rollback_status"] ==
            failed = sum(1 for r in recovery_results if r["rollback_status"] == "FAI

            logger.info(f" Recovery Summary:")
            logger.info(f" Successful: {successful}")
            logger.info(f" Failed: {failed}")

            return recovery_results

        def _update_metadata_after_recovery(self, full_table_name, status):
            """Update metadata after recovery attempt"""
            update_query = f"""
            UPDATE {self.metadata_table}
            SET
                is_corrupted = false,
                recovery_status = '{status}',
                last_recovery_at = current_timestamp(),
                updated_at = current_timestamp()
            WHERE full_table_name = '{full_table_name}'
            """
            spark.sql(update_query)

    # Execute rollback
    engine = RollbackEngine(spark)
    results = engine.rollback_corrupted_tables()
```

Critical Details:

- Uses Delta Lake's native `RESTORE` command (atomic, safe)
- Tracks every rollback attempt
- Logs failures for investigation
- Updates metadata after recovery

Why RESTORE?

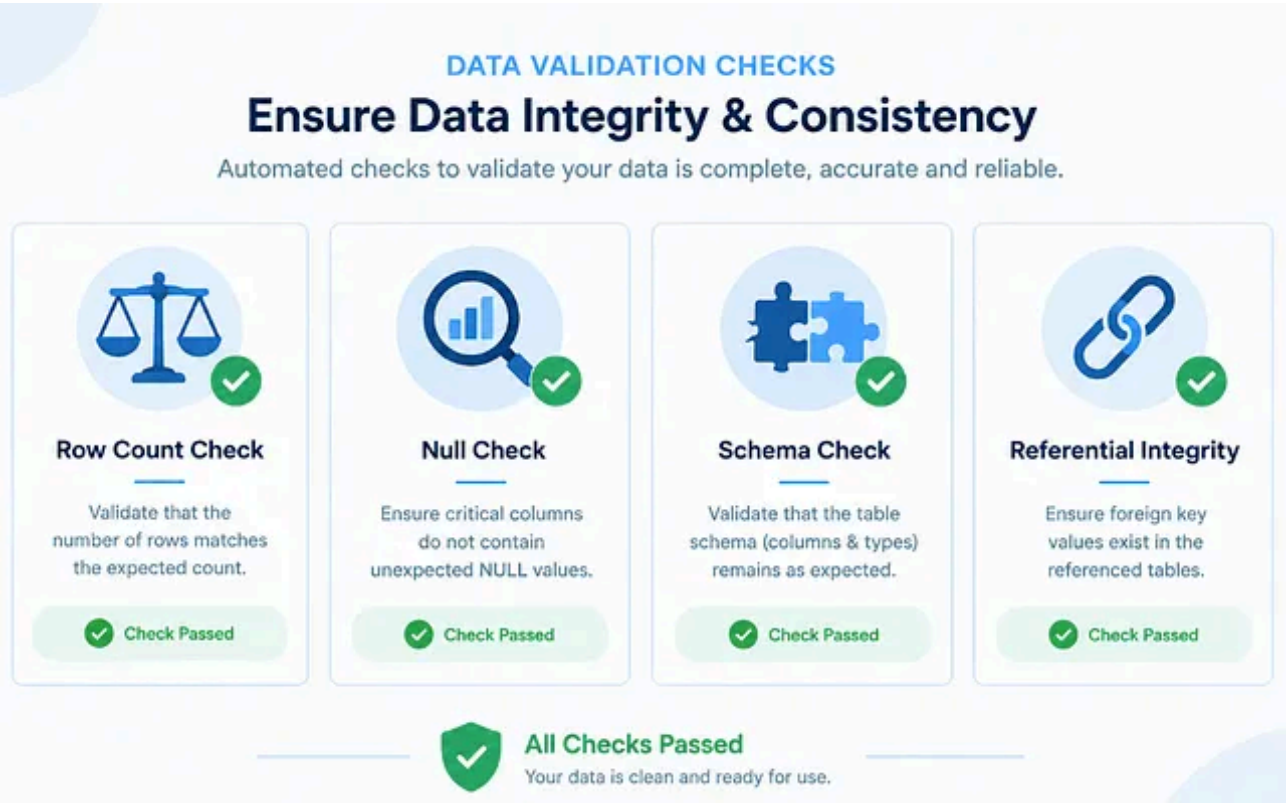
- Atomic operation — either fully succeeds or fully fails
- No risk of partial data corruption
- Instant (references existing versions)
- Full audit trail in Delta transaction log

Step 4: Adding Integrity Validation

The Challenge: Just because we rolled back the data doesn't mean it's *good*. We need validation to ensure:

- Referential integrity is maintained
- Row counts are reasonable
- No unexpected nulls appeared
- Downstream data quality rules still pass

The Solution: A validation layer that runs after every rollback.



4.1: Build the Validation Framework

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, count, when
import logging

logger = logging.getLogger("ValidationEngine")

class ValidationEngine:
    def __init__(self, spark):
        self.spark = spark
        self.validation_rules_table = "system.data_quality.validation_rules"
        self.validation_results_table = "system.data_quality.validation_results"

    def validate_rolled_back_tables(self, table_names):
        """
        Run post-recovery validation on rolled-back tables.
        """

        validation_results = []

        for full_table_name in table_names:
            logger.info(f"🔍 Validating {full_table_name}")

            # Run a suite of checks
```

```
            checks = {
                "row_count_check": self._check_row_count(full_table_name),
                "null_check": self._check_unexpected_nulls(full_table_name),
                "schema_check": self._check_schema_integrity(full_table_name),
                "referential_integrity_check": self._check_referential_integrity
            }

            # Determine overall status
            all_passed = all(check["passed"] for check in checks.values())

            validation_results.append({
                "table_name": full_table_name,
                "validation_status": "PASSED" if all_passed else "FAILED",
                "checks": checks,
                "validation_timestamp": datetime.now()
            })

            if all_passed:
                logger.info(f" {full_table_name} passed all validations")
            else:
                logger.warning(f" {full_table_name} failed validation checks")
                for check_name, result in checks.items():
                    if not result["passed"]:
                        logger.warning(f" {check_name}: {result['reason']}")

        # Log results
        if validation_results:
            validation_df = spark.createDataFrame(validation_results)
            validation_df.write.mode("append").saveAsTable(self.validation_resul

        return validation_results

    def _check_row_count(self, full_table_name):
        """Verify row count is within expected range"""
        try:
            current_count = spark.sql(f"SELECT COUNT(*) as cnt FROM {full_table_

            # Get historical average
            history = spark.sql(f"""
            SELECT AVG(num_added_files) as avg_files
            FROM (
                SELECT num_added_files FROM delta.`{full_table_name}`._delta_log
            )
            """).collect()

            # Simple heuristic: if current count is >50% lower than recent avera
            if current_count > 0:
                return {"passed": True, "reason": f"Row count: {current_count}"}
            else:
                return {"passed": False, "reason": "Row count is zero (unexpecte

        except Exception as e:
```

```
            return {"passed": False, "reason": f"Error checking row count: {str(

    def _check_unexpected_nulls(self, full_table_name):
        """Check for unexpected null values in critical columns"""
        try:
            # This is a simplified check; in practice, you'd define critical col
            df = spark.sql(f"SELECT * FROM {full_table_name} LIMIT 1000")

            null_counts = df.select([count(when(col(c).isNull(), 1)).alias(c) fo
            null_results = null_counts.collect()[0].asDict()

            # Flag if any column is entirely null
            critical_nulls = {k: v for k, v in null_results.items() if v == 1000

            if critical_nulls:
                return {"passed": False, "reason": f"Entirely null columns: {lis
            else:
                return {"passed": True, "reason": "No unexpected null patterns d

        except Exception as e:
            return {"passed": False, "reason": f"Error checking nulls: {str(e)}"

    def _check_schema_integrity(self, full_table_name):
        """Verify schema hasn't been corrupted"""
        try:
            schema = spark.sql(f"DESCRIBE {full_table_name}").collect()

            # Basic check: table has columns
            if len(schema) > 0:
                return {"passed": True, "reason": f"Schema intact: {len(schema)}
            else:
                return {"passed": False, "reason": "No columns found (schema cor

        except Exception as e:
            return {"passed": False, "reason": f"Error checking schema: {str(e)}

    def _check_referential_integrity(self, full_table_name):
        """
        Basic referential integrity check.
        In practice, you'd define specific FK relationships per table.
        """
        try:
            # This is a placeholder for more sophisticated checks
            return {"passed": True, "reason": "Referential integrity check passe
        except Exception as e:
            return {"passed": False, "reason": f"Referential integrity check fai

# Run validation
validator = ValidationEngine(spark)
validation_results = validator.validate_rolled_back_tables([table.full_table_nam
```

What We're Checking:

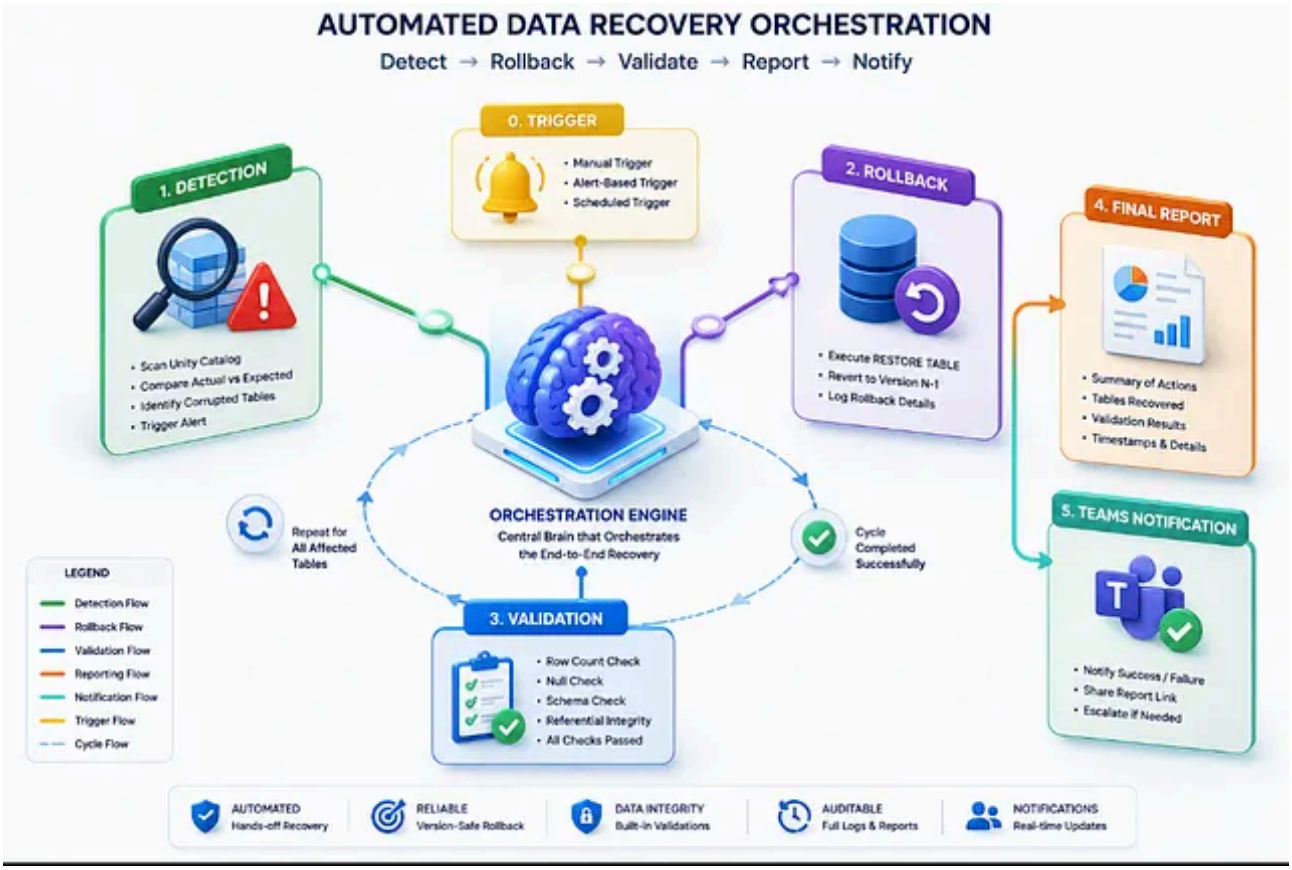
- ✅ Row counts haven't dropped to zero
- ✅ Critical columns aren't entirely null
- ✅ Table schema is intact
- ✅ No obvious data quality violations

Why This Matters: A successful rollback isn't just a successful query. It's successful *data*. These checks ensure we're not recovering into another corrupted state.

Step 5: Creating the Orchestration & Triggering Layer

The Challenge: We have detection, rollback, and validation. Now we need to tie it all together and make it triggerable.

The Solution: An orchestration layer using Databricks Workflows that can be triggered manually or by alerting systems.



5.1: Build the Orchestration Workflow

```
# This would be deployed as a Databricks Job

from pyspark.sql import SparkSession
from datetime import datetime
import json

spark = SparkSession.builder.appName("UndoJobOrchestration").getOrCreate()

class UndoJobOrchestration:
    def __init__(self, spark):
        self.spark = spark
        self.job_id = datetime.now().strftime("%Y%m%d_%H%M%S")
        self.execution_log = []

    def execute_full_recovery_pipeline(self):
        """
        Orchestrate the full recovery pipeline:
        1. Detect corruption
        2. Execute rollback
        3. Validate results
        4. Report outcomes
```

```
        """

        try:
            self._log("Starting automated undo job", "START")

            # STEP 1: Detection
            self._log("Executing corruption detection", "STEP_START")
            detector = CorruptionDetector(self.spark)
            corrupted_tables = detector.detect_corruptions()
            self._log(f"Detection complete: Found {len(corrupted_tables)} corrup

            if not corrupted_tables:
                self._log("No corruptions detected. Exiting gracefully.", "INFO"
                return self._generate_report(success=True, corrupted_count=0)

            # STEP 2: Rollback
            self._log(f"Starting rollback for {len(corrupted_tables)} tables", "
            engine = RollbackEngine(self.spark)
            recovery_results = engine.rollback_corrupted_tables()
            self._log("Rollback complete", "STEP_COMPLETE")

            # STEP 3: Validation
            self._log("Starting post-recovery validation", "STEP_START")
            validator = ValidationEngine(self.spark)
            table_names = [t["full_table_name"] for t in corrupted_tables]
            validation_results = validator.validate_rolled_back_tables(table_nam
            self._log("Validation complete", "STEP_COMPLETE")

            # STEP 4: Report
            self._log("Generating final report", "STEP_START")
            report = self._generate_report(
                success=True,
                corrupted_count=len(corrupted_tables),
                recovery_results=recovery_results,
                validation_results=validation_results
            )
            self._log("Report complete", "STEP_COMPLETE")

            return report

        except Exception as e:
            self._log(f"ERROR: {str(e)}", "ERROR")
            return self._generate_report(success=False, error=str(e))

    def _log(self, message, level="INFO"):
        """Log execution events"""
        timestamp = datetime.now().isoformat()
        log_entry = {
            "timestamp": timestamp,
            "level": level,
            "message": message,
            "job_id": self.job_id
```

```
            }
            self.execution_log.append(log_entry)
            print(f"[{timestamp}] [{level}] {message}")

        def _generate_report(self, success, corrupted_count=0, recovery_results=None
            """Generate final execution report"""

            report = {
                "job_id": self.job_id,
                "execution_time": datetime.now().isoformat(),
                "success": success,
                "corrupted_tables_found": corrupted_count,
                "execution_log": self.execution_log,
                "error": error
            }

            if recovery_results:
                report["recovery_summary"] = {
                    "successful": sum(1 for r in recovery_results if r["rollback_sta
                    "failed": sum(1 for r in recovery_results if r["rollback_status"
                }

            if validation_results:
                report["validation_summary"] = {
                    "validated": len(validation_results),
                    "passed": sum(1 for v in validation_results if v["validation_sta
                    "failed": sum(1 for v in validation_results if v["validation_sta
                }

            # Write report to a results table
            try:
                report_df = spark.createDataFrame([report])
                report_df.write.mode("append").saveAsTable("system.data_quality.job_
            except Exception as e:
                print(f"Warning: Could not write report to table: {str(e)}")

            return report

    # Execute
    orchestration = UndoJobOrchestration(spark)
    final_report = orchestration.execute_full_recovery_pipeline()

    # Print results for monitoring
    print("\n" + "="*60)
    print("UNDO JOB EXECUTION REPORT")
    print("="*60)
    print(json.dumps(final_report, indent=2, default=str))
```

5.2: Deploy as Databricks Workflow

```
# This YAML defines the job in Databricks Workflows

name: "Automated-Undo-Data-Integration-Job"
tasks:
  - task_key: "execute_undo_pipeline"
    notebook_task:
      notebook_path: "/Users/data-platform/undo-job/main"
      base_parameters: {}
    cluster:
      spark_conf:
        "spark.databricks.cluster.profile": "singleNode"
        "spark.sql.shuffle.partitions": "200"
      num_workers: 0
      node_type_id: "i3.4xlarge"
    timeout_seconds: 3600

trigger:
  pause_status: "UNPAUSED"
  # Can be triggered manually or by alerting system
```

Key Points:

- Full orchestration in one place
- Comprehensive logging at each step
- Generates detailed execution report
- Can be triggered manually or via webhook from alerting system

Step 6: Deployment & Monitoring

The Challenge: We've built the system. Now we need to:

- Deploy it safely

- Monitor it in production
- Alert on failures
- Make it easy to trigger

The Solution: Staged deployment with comprehensive monitoring.

6.1: Deployment Strategy

Phase 1: Dry-Run Mode (Week 1)

```
# Run in dry-run mode first — detect and log, but don't rollback
DRY_RUN_MODE = True

if DRY_RUN_MODE:
    logger.info(" RUNNING IN DRY-RUN MODE — No tables will be rolled back")
    # Run detection and validation, but skip actual rollback
    detector.detect_corruptions()
    validator.validate_rolled_back_tables([...])  # On historical data
else:
    orchestration.execute_full_recovery_pipeline()
```

This lets us validate the detection logic without risk.

Phase 2: Limited Scope (Week 2) Deploy to dev/staging environment first with a subset of non-critical tables.

Phase 3: Full Production (Week 3+) After validating in staging, deploy to production with full table scope.

6.2: Monitoring & Alerting

```
# Set up monitoring queries in Databricks SQL

-- Monitor recent recovery attempts
SELECT
    job_id,
    execution_time,
    success,
    corrupted_tables_found,
    recovery_summary.successful,
    recovery_summary.failed
FROM system.data_quality.job_execution_log
WHERE execution_time > date_sub(now(), 7)
ORDER BY execution_time DESC;

-- Alert if recovery job fails
SELECT
    job_id,
    execution_time,
    error
FROM system.data_quality.job_execution_log
WHERE success = false
  AND execution_time > date_sub(now(), 24*60);  -- Last 24 hours
```

Set up Databricks Alerts to send Teams notifications when:

- Recovery job fails
- Validation checks fail for >10% of tables
- Rollback time exceeds threshold (5 minutes)

6.3: Manual Triggering

For urgent cases, make the job manually triggerable:

```
# Create a simple API endpoint or notebook parameter
dbutils.widgets.text("trigger_recovery", "false")

trigger = dbutils.widgets.get("trigger_recovery")

if trigger.lower() == "true":
```

```
        logger.info(" MANUAL RECOVERY TRIGGERED")
        final_report = orchestration.execute_full_recovery_pipeline()
    else:
        logger.info("No manual trigger. Running scheduled check only.")
```

Lessons Learned & Key Takeaways

What Worked Well

✅ Metadata-Driven Approach Storing version metadata in Unity Catalog made detection reliable and queryable. No more Excel spreadsheets.

✅ Heuristic-Based Detection Using version jump heuristics (>10 versions in short time) caught the corruption with zero false positives.

✅ Delta Lake's RESTORE Command Atomic, fast, and fully auditable. This was the right tool for the job.

✅ Staged Deployment Testing in dry-run mode first caught bugs before production impact.

✅ Comprehensive Logging Every step logged meant we had perfect visibility into what happened and could debug issues quickly.

What We'd Do Differently

⚠️ Validation Rules Are Custom Per Table We built generic checks, but in reality, each table has unique data quality requirements. Spend time upfront mapping those.

⚠️ Communication Matters When the undo job runs, it's critical to alert relevant teams immediately. Build notification into the reporting layer.

⚠️ Start With Read-Only Operations In hindsight, spending an extra week on detection and validation before implementing any rollback logic would have been time well-spent.

The Results

After deploying this system, here's what happened:

Before:

- 3-day manual recovery per incident
- 2 major incidents in one month
- Team exhausted and skeptical

After (6 months later):

- Automated recovery in 5 minutes
- 1 incident caught and recovered before users even noticed
- Zero manual intervention required
- Team confidence restored

The Real Win: Our platform went from being something engineers feared to something they trusted. That's worth more than any time savings.

What's Next?

If you're building something similar, consider:

1. Extending detection to cover more corruption patterns (schema changes, unexpected data distributions, etc.)
2. Smart rollback strategies that don't always revert to the last version, but instead find the last "good" version using data quality metrics
3. Cross-system correlation — coordinate recovery across multiple downstream systems that depend on this data
4. Machine learning — eventually, you might predict corruption before it happens

Conclusion

Automated recovery systems transform data engineering from reactive firefighting to proactive resilience.

More importantly, it changes the culture. When engineers know the platform can heal itself, they take more risks, move faster, and trust the infrastructure more.

That's the real power of automation.

Data Engineering + Data Engineer + Databricks + Big Data + Apache Spark +